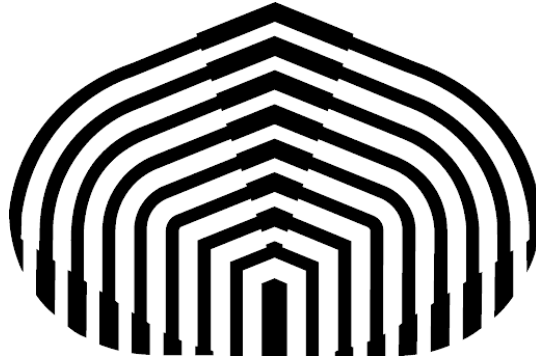




Universidad Simón Bolívar
Departamento de Computación y Tecnología de la Información
Lenguajes de Programación I (CI-3641)
Examen 4
Miguel Salomon - 1910274

1. Asociación de métodos y herencia (6 pts)

Análisis de clases:

- A: $x=2$, $f(y) = 2+y$
- B extends A: $x=5$, $f(y) = 5*y$
- C extends B: $g(z) = f(z+x)$ (Hereda f de B)

Objetos:

- A $p = \text{new B}()$;
- B $q = \text{new C}()$;
- A $r = \text{new C}()$;

a) Salida con asociación estática de métodos y resolución estática de campos

El método y la variable se determinan por el tipo de la variable declarada.

- p (Tipo A): Llama a $A.f(2)$. $A.x$ es 2. Retorna $2 + 2 = 4$.
- q (Tipo B): Llama a $B.f(2)$. $B.x$ es 5. Retorna $5 * 2 = 10$.
- r (Tipo A): Llama a $A.f(2)$. $A.x$ es 2. Retorna $2 + 2 = 4$.

Total: $4 + 10 + 4 = 18$.

b) Salida con asociación dinámica de métodos y ocultamiento de campos

El método se determina por el tipo del objeto (new ...). Los campos se resuelven dentro del contexto de la clase que posee el método ejecutado.

- p (Objeto B): Llama a B.f(2). Dentro de B, x refiere a B.x (5). Retorna $5 * 2 = 10$.
- q (Objeto C): Hereda f de B. Llama a B.f(2). Usa B.x (5). Retorna $5 * 2 = 10$.
- r (Objeto C): Hereda f de B. Llama a B.f(2). Usa B.x (5). Retorna $5 * 2 = 10$.

Total: $10 + 10 + 10 = 30$.

c) Pila de ejecución para r.f(2)

Orden de apilamiento (crecimiento hacia abajo):

Componente	Valor / Descripción
Argumento y	2
Puntero this	Referencia al objeto C en el Heap
Variables Locales	Ninguna en este método
Dirección de Retorno	Dirección en main donde se llamó a r.f(2)
Enlace Dinámico	Puntero al registro de activación del llamador, e decir, main

2. Pasaje de parámetros (6 pts)

Estado inicial global: $g = 1$. Llamada: $\text{calc}(g, g, g)$.

1. a es alias de g (1). c es alias de g (1). b es copia (1).

2. $a = a+b \rightarrow g = g+1 \rightarrow g=2$. $\text{Print}(2)$.

3. $b = b+c \rightarrow b = 1+g \rightarrow b = 1+2 = 3$. (Local b cambia, g no).

4. $g = g+a+b \rightarrow g = g+g+b \rightarrow g = 2+2+3 = 7$

1. b es alias de g (1). a, c son copias (1).

2. $a = a+b \rightarrow a = 1+1 = 2$. g no cambia. $\text{Print}(1)$.

3. $b = b+c \rightarrow g = g+1 \rightarrow g = 1+1 = 2$.

4. $g = g+a+b \rightarrow g = g+2+g \rightarrow g = 2+2+2 = 6$

1. c es alias de g (1). a, b son copias (1).

2. $a = a+b \rightarrow a = 2$. g no cambia. $\text{Print}(1)$.

3. $b = b+c \rightarrow b = 1+g \rightarrow b = 1+1 = 2$.

4. $g = g+a+b \rightarrow g = 1+2+2 = 5$

a	b	c	Salida
Referencia	Valor	Referencia	7
Valor	Referencia	Valor	6
Valor	Valor	Referencia	5

3. Orden de evaluación e infinitud (6 pts)

Expresión: `weird ["u", "v"] (inf 1)`

Donde `'inf 1'` genera `[1, 2, 3, ...]`.

a) Evaluación Normal

En orden normal, los argumentos no se evalúan hasta que son necesarios. La función `'weird'` usa `'foldr'`.

La expansión ocurre 'de afuera hacia adentro'. El `'foldr'` comienza a consumir la lista `["u", "v"]`.

1. `weird` expande a: `pairAll "u" (pairAll "v" (inf 1))`
2. `pairAll "u" ...` necesita evaluar su segundo argumento sólo parcialmente para producir la cabeza de la lista resultante.

El programa produce una lista infinita de pares o tuplas anidadas (dependiendo de la interpretación exacta de `pairAll`) de forma incremental. No se cuelga inicialmente por lo que produce salida.

b) Evaluación Aplicativa

En orden aplicativo, los argumentos de la función se evalúan completamente antes de entrar al cuerpo de la función.

1. Se intenta evaluar el argumento `(inf 1)`.
2. `(inf 1)` genera `1 : 2 : 3 ...` infinitamente.

El programa diverge tratando de construir la lista completa de números antes de llamar a `'weird'`. Nunca produce salida.

c) Divergencia

La divergencia aparece en el caso (b) debido a la evaluación estricta de una estructura de datos infinita pasada como argumento.

4. Fold sobre árboles (6 pts)

a) Definición de foldT

$foldT :: (a \rightarrow b \rightarrow b \rightarrow b) \rightarrow b \rightarrow Tree\ a \rightarrow b$

$foldT\ f\ base\ Leaf = base$

$foldT\ f\ base\ (Node\ x\ l\ r) = f\ x\ (foldT\ f\ base\ l)\ (foldT\ f\ base\ r)$

b) Definición de label usando foldT

Asumiendo que 'label' busca si un elemento existe en el árbol:

$label\ k\ t = foldT\ (\backslash x\ l\ r \rightarrow x == k\ ||\ l\ ||\ r)\ False\ t$

c) Evaluación Parcial y Estrategias

Árbol $tInf\ n = Node\ n\ (tInf\ (n+1))\ (tInf\ (2*n))$

Expresión: $label\ 0\ (tInf\ 1)$

Evaluación Normal:

El operador $(||)$ es no estricto en el segundo argumento si el primero es True. Pero aquí, 'x == k' siempre es False ya que 0 no está en los positivos.

Traza: $1==0\ (False)\ ||\ label\ 0\ (tInf\ 2)\ ||\ \dots$

Haskell intentará evaluar la rama izquierda recursivamente buscando el 0. Como el árbol es infinito y 0 no existe, descenderá infinitamente por la izquierda por lo que diverge.

Evaluación Aplicativa:

Evalúa los argumentos de 'label' primero. Intenta construir 'tInf 1' completo. Como es infinito, diverge inmediatamente antes de llamar a label.

5. Gramáticas (6 pts)

Gramática ambigua.

a) Por qué es ambigua

La sentencia: 'if E then if E then S else S' tiene dos árboles de análisis sintáctico posibles:

1. El 'else' pertenece al 'if' interno.
2. El 'else' pertenece al 'if' externo.

b) Reescritura sin ambigüedad

Un 'if' abierto sin else no puede estar antes de un 'else'.

S -> Matched | Open

Matched -> if E then Matched else Matched | print

Open -> if E then S | if E then Matched else Open

c) Derivación izquierda

String: if true then print else if false then print else print

Usando la gramática corregida S -> Matched, ya que termina balanceado:

-> S

-> Matched

-> if E then Matched else Matched

-> if true then Matched else Matched

-> if true then print else Matched

-> if true then print else (if E then Matched else Matched)

-> if true then print else (if false then Matched else Matched)

-> if true then print else (if false then print else Matched)

-> if true then print else (if false then print else print)